

ActInf OrgStream 009.1 ~ The Value of Open Source Software (Hoffmann, Nagle, Zhou)

OrgStream | The Value of Open Source Software (Hoffmann, Nagle, Zhou) | 1:07:50

Hello, welcome.

It is June 18th, 2025, and but your cover slide had me guessing for sure.

And we're in Active OrgStream 9.1 with Manuel Hoffman and with Bradley Alicia as guest.

We will be discussing their recent paper, The Value of Open Source Software.

And looking forward to this, a lot to say that I'm sure will come after your presentation.

So thank you again for joining and to you for the presentation.

Great. Yeah, fantastic.

Thank you, Daniel, for inviting me to present this work.

And yeah, Bradley, looking forward to the discussion and everybody else on YouTube as well.

If you have any comments, obviously, that's much appreciated.

All right.

Well, so let me just dive right into this, right?

So we're covering here the value of open source software.

Let me just give you an idea, right?

And probably you are closer to that than maybe some of the other audience members to whom I usually present.

But you can think about a bunch of companies here, Google, Ford, Intel, John Deere.

And we usually can just ask, you know, what do they have in common?

Well, actually, what they have in common is that, among others, they are really examples of users and contributors to open source software.

And we know that open source software is really a cornerstone of modern software development.

So 96% of firm code bases contain some form of open source software.

That's according to the software analytics company Synopsis.

And 76% of all code in firm code bases is actually open source software code.

It's hugely important in cutting edge technologies, artificial intelligence, quantum computing, big data and analytics, and many other areas.

So one venture capitalist, Mark Anderson, once said software is eating the world.

And that was enhanced by Joseph Jacks, who used that thing, software, open source is actually eating software faster than software is eating the world.

And that's why it's not.

Just talking about the dominance of open source software, which just to give you the rough idea of the definition, right?

Open source software is software whose source code is publicly available, usually under some permissive license.

In particular, we are interested in free open source software very often, which is a public good.

It's generally non-rival.

So if I consume it, there's not less available for you, and it's usually non-excludable.

So basically, it's the ideal form of a public good that everybody can consume.

And often you can really think of that as a global public good like education.

Individuals and firms have used open source software and contribute to it substantially, right?

So there's been a bunch of literature, especially by my co-author here, Frank Nagle, but also a lot of other people.

And we are continuously working on better understanding the open source software ecosystem, companies and individuals that engage in it.

So now you're probably very familiar with this slide.

Maybe I've seen it too often, this kind of figure, right?

Where we can see this huge machinery of our modern digital infrastructure.

And then there is this little piece at the bottom, which is a project by some random person in Nebraska that has been thanklessly maintained since 2003.

So what that tells you, right, there is first, there is some huge dependency on some highly active individual, some open source maintainers without our without there could be a potential collapse of our modern digital infrastructure.

And on top of that, there is a issue of the common, which is that which is common to the greatest number gets the least amount of care, according to Aristoteles.

Right. So there's some kind of free riding potential, not only, but also, I think, issues that we observe anecdotally and which we also see here in the data as well.

So now, why do we care about the value of open source software?

Well, I guess we care about the value of a lot of goods.

But here in particular, we do not know actually what's the value of open source software, because it doesn't even come up in traditional gross domestic product measures, right, that are in accounting systems.

And also some firms may just not be aware of productivity and cost saving benefits.

So that's why we are bringing out this paper to better understand the value of open source software, measuring the value here in a very simple way, as the price times quantity $P \times Q$.

And so here, open source software, you have really the problem, the price of open source software is zero, right?

There is no price that is the actual measurement problem.

And the quantity, well, it's infinitely replicable, because the license allows for free copying and usage is not easily trackable.

So we have a problem on the price side, we have a problem on the quantity side, and we're not saying that we're going to solve this problem, but we're saying we're going to try to make some inroads here.

So that's in the spirit of what Warren Buffett said, price is what you pay and value is what you get.

So right, we price could be one idea of value, but it's really not the actual value.

And of course, we want to provide a value measure here, because usually a lot of folks, managerial folks, firms working in companies, but really anybody wants to understand, wants to get an idea of how valuable something is.

So, because otherwise, people may not allocate resources towards it, or do not really understand the

importance of it.

So, we ask, how much would we lose, really, if open source software did not exist?

Or ask differently, how much would it cost us to replicate the universe of open source code?

And for prices, we're going to calculate the prices using what we like to call a replacement value, or so called, you could also use purchase price.

And for quantity, we will use two large data sets, the largest ones that we are aware of.

One is the census of open source software, which is inward facing towards within the company, the products that they are using and then reselling, and then built with, which is an outward facing data set, that is basically based, it's based on websites and the usage of open source software and websites.

Now, to measure the value of open source software, you could, or really the value of anything that is difficult to measure, you could go about it in two ways.

You could go about it from the output market side.

The output market would be the goods market from the firm's perspective.

And you can just, let's say you have a good that has a price of zero, then you'll find a substitute good that actually has a positive price.

And you may be able to impute that to get an understanding of what the value may be.

And North House has famously made this very general case for greenhouse gas emissions.

And Greenstein-Nagel have, for example, calculated the value of Apache servers being around \$2 billion.

And that has been extended with web servers in general, the value being around \$4.5 billion.

And they could do that because they have had some information on what the substitute good was.

And so that is very helpful, right?

So they have information on usage of all of the web servers.

But then you can only really look at one particular type of software here, web servers.

And it's difficult to expand that because you would need to find a basket of goods, a gigantic basket for each of the open source software packages.

And some are not, you know, are really small pieces, some are larger pieces.

It's difficult to find those substitute goods.

So what we did is we're going to use the second approach, the input market approach or the labor approach.

So from a firm's perspective, you're going on the labor market and we're going to look at the replacement value via labor costs.

You can think about that in the spirit of household innovations by Sickle and Hipple.

The main idea here really is let's think about the housework that you are doing, right?

You're cleaning something, you're vacuuming in your house and you're not getting paid for that, right?

You just do that in your free time.

That's not going to add to GDP, but you could instead hire somebody and they could do the work and vacuum for you.

And that would be the price that it would cost to do to engage in this household activity.

And then you could measure the value.

And we are going to do a similar exercise.

And we're not the first one here.

There is a blind and co-authors that have done a kind of similar exercise for the European Union.

Robbins and co-authors have done an exercise for the United States where they estimated the supply side value being around 1 billion to \$38 billion.

We do this not on the European or US level, but we think about that as a global estimate.

But we are going to cut down and we're going to look at firm specific contributions.

So we're going to discard kind of the long tails of contributions that may happen from individual small contributions.

So that's where our estimates may differ.

So now the good thing about this approach is we have all of open source.

We can use all of open source software and we can assign some kind of price by imputing the wage that it would cost to create this product.

And the negative aspect is maybe it's difficult to get the usage information.

But now we're going to we are also approximating usage information through our two large data sets.

So now for our methodology, we're using really multiple counterfactuals.

Well, we are mainly using method two, but in one counterfactual, we're also going to use a part of method one a little bit.

And I'm going to explain this in more detail here.

So our first counterfactual is the supply side counterfactual.

So it's the idea that open source software as a concept just purely doesn't exist.

And now the whole code has to be rewritten once.

Now we're doing that by looking at all open source packages that are used by companies in particular.

And we think about addressing production externalities.

So like we are just holding constant people that are working one company are not going to move to another company and approve or like are not benefiting from the production of one piece of code when they are then producing another piece of code.

But they are just producing those pieces of code separately.

Then we can't count the lines of codes per package and calculate the labor replacement value.

So which is in this case, the value of from the supply side.

And we can dive into a little bit how this labor replacement cost value works exactly after going through those three counterfactuals.

Now, the second counterfactual is, well, if the concept of open source doesn't exist anymore.

Now, what would happen is maybe one firm would re-write, would emerge and re-write the code and then sell it to all of the other firms.

So that's now we would just think about obtaining the information usage information that we have from both data sets on the packages.

We address consumption externalities holding so there are no additional spillovers.

Then we sell the one seller, creates all of those packages and then they sell it at the market price P .

So here you can really see it's just the price times quantity over all of the packages minus the fixed costs that

it takes to generate those packages,

which is really as a value from the supply side from the first counterfactual and the elasticity of demand.

Here is around one. We did some that we set around one based on the literature, but we estimated it and varied it.

So engage in some robustness checks that you can find in the papers as well.

And so the main idea is that we find an optimal price P here, give them all of the quantities, and then we can create,

then we can get a measure of the overall value from a one seller demand side.

Now, a third counterfactual is what we like to call the firm automatically demand side.

And this is the concept of open source software. Again, doesn't exist anymore.

Now, each firm has to rewrite the code by themselves.

We again obtain all of the usage information, address consumption externalities, but now there's a single replacement idea within a firm.

So when you use a product, yes, you can use it multiple times a package, but we are really thinking of that.

We don't have to generate that. There's no additional value that is being generated from using it additional times because you're just using.

You can just freely move it within the company and there's no price for internally using it.

Now we calculate the value here by using the supply side value and multiplying it by all the times that it has been used each package for each of those companies.

So, all right.

All right. So what is the data about?

Well, we have the census two or three and open source software, which has been a really mega project, I would say by my co-author, Frank Nagel at HBS.

And this is from 2020 data from 2020 in particular, there is a third version coming up and they are continuously a new version.

And we are trying to see how we can get more information about open source software usage that is particularly firm related.

And here you can see code usage of firms globally. So they're around 600,000 packages, but we are going to really home into the top packages.

So we are cutting off a huge tail on the right side.

This is information from software composition analysis firms.

So they are scans. What those companies here, Synopsys and Snyk, for example, or Forsa are doing is they usually ask to scan all of the usage of open source software and software in general within firms.

For example, when a merger is happening or before a merger is happening because the companies want to know what kind of potential liability exists.

And so they really check for license compliance.

Now, the second data. So this is the kind of inboard facing data sets that we construct.

And the second complimentary data set is from build with, which is a service which basically scans all of the websites globally and gets the code from all of them.

And then it measures all of the technologies that are embedded in those websites.

And among others, this control contains all open source software web development.

also called Kokomo from the US Department of Defense,
to estimate software project cost.

It's quite a flexible model,
but we are just, we are going ahead with the default parameters,
but again, we are engaging in some robustness checks
with those parameters as well.

So really here, it's,
here the effort stands for person month effort,
so how many people do you need per month?

And then LOC stands for the lines of codes
and thousands of lines of codes.

And we just have some parameters here that we,
that are nonlinear transformation that we,
that we can vary, but we use a default for this exercise.

And one aspect is, well, once we know, you know,
how many people we need, then we also need to provide,
give an idea about what the wage
for those individuals may be, right?

And so from salary expert, another database,
we got the monthly growth salary
with the range from 3000 to \$10,000.

So from a low and high wage country,
let's see if this button works properly.

So here you can see the top 30 countries
that are included for what we call the global wage.

So we included 80%, 88% of GitHub activity from 2020.
And we used all of those companies in the global,
in a global wage estimate.

So we have really have three estimates.

One is you like an estimate
where we used a low wage from India,
one where we used a high wage from the US
and something in between with all of those countries
that I just show you as a global average wage.

All right.

Now, to give you a sense of the data,
here are some descriptive statistics for both panel A,
which is the census and built with data in panel B.

So outward and, or inward and outward facing.
So you can see the lines of codes for all packages
in particular, here's a number of,
like the lines of codes is roughly maybe 70% of that.
When we just focus on the top five languages,
and we can see the usage of all packages
around 2.7 million times that packages were used here,
the around 1,800 packages,
or for the top five languages, around 1,600 packages.
BuildWords has a little bit,
has also roughly the same ratio of top five languages
that we identified based on GitHub's top five languages
during that time period.
And I show you what those were
when I go through the results,
but those are around 740 projects.
All right.
So now,
here's basically the main finding, right?
If you wanna take anything away from this,
it's here, the value of open source based on pure code.
So we're getting rid of any kind of addition
of what could be data, for example,
or could be maybe machine generated.
And what we find is for a low wage country,
as would be the Indian wage,
we found around a \$1 billion value on the supply side
of firm related open source software packages,
where there's a global estimated around 4 billion
and 6 billion on the higher side for US wage,
as a value.
Now, when you have one seller that is taking,
and that is recreating everything
and then selling the packages to those different firms
that are using it,
we have almost,
we have basically a constant value of 5 point,
around \$5 trillion.

There is some variation,
but basically the wages here don't matter as much
because they're in the fixed cost.

And then finally,
when we assume that all of those companies
have to replace open source software packages
by themselves internally,
and they don't resell products to other firms,
then we get a value of around \$88 trillion globally
in this case, right?

So you can see the,
look at this as some kind of upper bound,
and maybe we may be in between this one seller case
and the firm Autarchy case in reality.

Now,
well,
let's take the 5.25 trillion
as kind of a lower bound estimate,
right?

Accounting in particular for usage,
so not just the supply side of your availability of packages.
And so this is around,
right?

5.25 trillion in demand side value over the time,
in particular measured in the year 2020.

And we know that private firms spent around 45% of software
cost in the year 2020.

And we know that private firms spent around 45% of software
cost in the year 2020.

And so this is around 5.25 trillion.
per cost on pre-package software.

So proprietary software,
right?

So now let's assume this kind of spending that is actually
us estimate holds globally.

This would be around a \$2.36 trillion of proprietary spending
on closed software if you like.

Now you add the value of the 5.25 trillion to that

at around \$7.61 trillion,

just purely in software cost.

So what that implies is that firms now would spend 3.2 times more on software than what they currently are spending in relation to,

because open source software, the concept just purely doesn't exist anymore in this world, right?

So that's a huge kind of cost saving for them

when you just think about open source software being freely available.

Now, we know there's a substantial value.

So one question that is quite natural here is how evenly is this actually distributed?

Well, and here are some examples of, well, this is here the top five languages plus Go from GitHub.

And you can see this is a supply side value of just recreating all of open source software once of the top five languages.

There are some substantial variation where Go and Java, you know, have quite a high value.

Python, for example, has a relatively low value type code.

JavaScript also relatively low value on the supply side.

C is somewhere in the middle.

So now this is not surprising.

This is more of a robustness check on the supply side for, so this was a census data set, the inward facing, the outward facing built with data set.

We conditioned already on JavaScript, which is often also related to TypeScript.

And so we picked up basically all of that variation in the external facing data set.

Now, for the census of open source software, unfortunately, we don't have any information on the heterogeneity by industry.

So we try to think about that a little bit more in the context of the outward facing built with data set.

And we got some, we could show you some heterogeneity in the demand side value of the trillions of dollars.

So we can see here an administrative and support and waste is quite high.

So we see this is kind of high interest in the interest in economics, we see this is kind of high interest in economics,

we see this is kind of high interest in economics, we see this is kind of high interest in economics,

Now you see manufacturing interestingly

is somewhere in the middle, I would say,

management of companies, enterprising, mining,

and public administration, utilities, or agriculture
is the value is relatively limited.

And to some degree that may be just of lower,
just substantially lower usage in those industries.

Now, how is this value distributed?

Or I guess, where does this value come from
when we think about the developers here?

And we plotted a Lorentz curve.

So with this idea of a Gini coefficient of being one,
right, if you have this, if you think about the Lorentz curve

here as tracing the 45 degree line,

this is a share of the value on the supply side

is this Y axis on the X axis,

you have the cumulative share of developers

that are creating this value,

and they're creating, they're engaged in repositories.

Here, the value is in blue,

and the engagement repositories in orange.

Now, if this blue line would be exactly aligning

with the 45 degree line,

that would mean that every developer contributes evenly.

it contributes equally really to the value share.

But we don't see that.

We see that this blue line here,

that is the value line for the share of developers,

is actually very close to the corner here,

and very far away from the 45 degree line.

And that's what we would expect also from the literature, right?

that there is a share of developers that is here,

maybe around five or 10% of individuals

that are creating a substantial value.

They are generating most of the value here on the supply side,
creating the packages.

And interesting, so interesting part here is that they are not just active
in a few projects, but they are active here in yellow,

in many, many different projects on average.

Now, we could go through the one seller demand side.

We actually see a very similar picture here.

And we see a huge inequality of just a few developers contributing a substantial amount to the value share.

And we see this again when we look at the firm Autarchy case.

So now, to conclude here from the inequality story,

5% of open source software developers will equate 93% of the supply side value,

and as a bound, 99% of the firm Autarchy demand side value.

So just the 5% of developers create a huge amount of value on the supply and the demand side.

Now, what about the firm, the value that firms,

that is kind of firm affiliated, right, that potentially firms are creating?

Well, what we find is that 23% of projects are firm affiliated.

Well, that implies that 77% are not.

Now, firm contributions to the value of open source software look as follows.

We see the share is around 6% of the firm contribution to on the supply side.

So on the one seller and firm Autarchy case, it's between 0.7% and 2.8%.

So the firm contributions here are as measured by the firm affiliation,

which may be an underestimate here, right?

But it's actually quite low.

The interesting aspect is that they supply more than they demand based on those usage statistics.

Also very interesting, if you think about, you know, individuals here certainly create a lot of value in this space,

but firms are also doing valuable things.

When you just think about correlations of firm affiliated repositories here in like reddish,

and then in kind of like this blue color, you see non-firm affiliated,

the shape of the density distribution for the days from the last commit measure,

which implies here that firms are really substantially more active.

the distribution is very close to relatively more close to zero than for individuals.

And that may imply that firm projects are just more actively maintained.

And that is overall a good thing.

Now, there are a bunch of reasons why we may think that what we are finding here is the value of open source software,

that we are actually underestimating the value.

The global data that we have is very particular global data from those software composition analysis firms, which is as good as what we could get as data, but the, you know, global data is still incomplete.

We cut off the tail of the distribution as mentioned.

We also got rid of comments, documentation, empty lines, anything that is not pure code.

We have an additional estimate for that in the paper.

Also, you can think about code being improved over multiple versions, right?

We are not accounting for that.

And that is, but that is also costly.

And you could also imagine, well, there are a bunch of projects that a company has to work on, and not all of the projects may work out.

But so it may be a necessary process to have a lot of project failure, with one successful project, but that implies that there are also costs that are associated with those kind of failures.

And those costs are not included here.

Now, we also think that even the highest wage scenario is kind of a lower bound on wages, because if you think about the individual that some of the individuals that are active in this space, they can obtain especially highly skilled individuals in open source, they can obtain quite high wages from some of the big tech companies.

Now, further, we just don't have the Linux kernel in here, which is probably a huge value driver when it comes to open source software.

Now, to conclude, the value of open source software on the supply side is around \$4 billion, with a one seller demand side value of \$5 trillion, which can go up to \$80 trillion when firms are just recreating codes by themselves.

So, in terms of the value of open source software, we find heterogeneity across industries and developers, where 5% of developed open source software developers create around at least 93% of the value.

And firms here make up really a small share, but projects are a bit more actively maintained by, or projects by firms are more actively maintained than by individuals.

So, without open source software, there would be really a large loss in value.

Firms would need to spend 3.2 times more on software than they currently do.

Thank you so much.

Thank you.

Awesome. Lot there.

Big iceberg. Only small amount showing.

Right.

Perhaps first, Bradley, maybe introduce and coming from the open source sides that you've worked on.

Like, where does this take you?

Where does this reflect on you?

Yeah.

So, you know, I come from a bit of a different perspective.

I do a lot of open source education and project management, and then I've worked in open source, not in terms of how corporations interface with it so much, but how basically you can do scientific research with open source and other types of things like, you know, creating games with open source.

And I like that you had here.

And I like that you had here.

I think that was a very good analysis of looking at like, sort of putting a value on, you know, what we have out there.

And, but I wonder, especially with respect to like the active inference aspect, you know, is it that people are motivated by money?

Or is it that they're in some cases, they're motivated by sort of a need in the in the communities are working in, and in the sort of the, you know, because a lot of like, if we think about Linux, and some of the things that have been created there was largely because there was some utility of a tool that they needed.

And there wasn't a tool anywhere else. So they created the tool, and then they made it into this thing that was distributed. And then it went from there. So the original motivation was actually more kind of like, almost like maybe reinforcement learning or something like that.

And then you know, when you get into when once it's distributed, then you have this financial imperative that might take over or this sort of cost imperative, like, because I know you showed that figure of the maintainers, and how they play this critical role in open source projects, and you know, a lot of the problem there stems from people not have it, you know, kind of doing it on the side, and then not having enough time to devote to actually

maintaining these large packages. So you have these vulnerabilities, you know, it may start that they love the project, and they're motivated by self interest, but then over time, you get, you know, a project that's so big that maintainers just simply can't afford to do that, you know, to spend like 70 hours a week maintaining this project.

So I mean, you know, I wonder about that, like, what are your thoughts about different types of motivations for open source production?

Yeah, no, I think you're absolutely right. Like the motivations, especially as you know, like the origin of open source, I think even what is created right now, as open source software tools, is very often, yes, there, we need some kind of tool, we don't have it.

And so let's create it. And I guess that's the origin of a lot of innovation, just generally speaking, like, or that's one of the idea of innovations, right? Like, I don't have this particular thing, I have a need and now, well, if nobody else is doing this, I'm going to do it. But they are willing to also share this, you know, like the open source community, to share things openly, that's the whole spirit of it. So yeah, I think, non peculiar

non peculiarly benefit, like non peculiarly motives are a huge driver of what is going on, how open source software exists in the first place, right. And this idea of sharing and supporting a community potential even like reputation building. And there are some, you know, aspects, of course, where this then, like, like, it's also kind of like a skill building exercise, but maybe people don't do that necessarily just for that purpose.

For that purpose, but it comes with it, then the like, once you create those kind of projects and are really successful in it, yes, you point out, then maybe you don't have the time anymore to even be able to maintain those projects just for free.

But then you have learned a substantial amount on the way, which is quite, is valued quite highly in the marketplace. So companies, I think do value that and we have seen papers around that where people even try to signal, hey, like I'm contributing to, you know, like, open source by engaging in a couple of commits before I'm getting hired. And then after that, you can see kind of like a drop off in the engagement.

But I think, yeah, there are many different motives, but certainly the non-pecuniary motives is quite a critical one in this environment. I completely agree.

Yeah.

Yeah.

So I mean, the other, oh, go ahead.

Go ahead.

Oh, yeah.

So the other thing I wanted to mention is that like, you know, the way kind of we do open source is kind of dependent on certain tools that we have. So, you know, we have forks of repositories.

And, you know, that makes it easy for people to say copy a repository and make, modify it and distribute it. And it's really interesting. I wonder if, you know, people thought about like, the economic sort of benefits of a lot of the innovations of version control where you have like forking, and you have like, you know, being able to control different versions and how they're distributed.

Because that kind of plays a key in like how people get value out of it. If I see a project, I want to fork it. I want to create like, I want to basically add value to it, change it so that it's useful to my field or my group.

And then, you know, if 1000 people do that, that's a lot of value out there. And if you can't do that easily, then the value is limited because it makes it harder to sort of keep all the distributions maintained and keep them all consistent.

So, you know, there are all these tools that kind of showed, you know, were kind of before we had the kind of version control we have now, it's probably much harder to get value out of the open source software.

It was just kind of like people kind of distributing it maybe on disks or, you know, their earlier versions of version control. And it would be very interesting to see where that sort of started up.

If that, if there were like the set of innovations that we have, say with GitHub, played a huge role and kind of like kickstarting a lot of the value open source.

Yeah, that's a great point. And we are in constant communication with the guys at GitHub and have some other work with the people at GitHub and generally, yeah, I agree with that also, like the benefits of version control, I would think are enormous.

I mean, they just contribute to this, like, like, outsourcing in general, right? But I haven't seen a paper on that. So because I guess it's difficult. Like, yes, we can look into the past.

And there will be getting value out of it.

Okay, sorry. I think you're frozen.

Okay, now you're back.

Yeah, yeah.

Okay.

Yeah, so just right. So you mentioned this point of disks, you know, like, code being distributed on a disk versus now with the later version control, it's the ones where everybody can access it. And I think this would be, it would be very useful to understand the world where we just have disks versus the world where we just have, where we have the version control that we have now, but have those two worlds right next to each other.

And it's, it's as difficult as our exercise that we did in this paper, because we have to imagine different kinds of counterfactual worlds, and then try to impose some assumptions.

I'm sure this can be done, or maybe there's a clever way of getting an understanding of what, you know, the benefit of version control is, but I'm sure that just the pure number of people that are getting pulled in, right?

You can, you're just building on, you know, pulling in the crowd substantially more and making it more convenient for everybody to contribute in a very easy way.

And that's why open source software is, I think, also successful because of, yeah, crowdsourcing.

And, yeah, you're just very easily, there's a zero cost basically from you replicating a package.

Yeah, just to this last point, we're still very generationally early in the global internet facilitated exchange of open source software.

So it's kind of like looking at the early developmental phases, and there's a lot of core technologies like Git and services like GitHub.

And there may be other facilitating technologies to open source overall, some that people are probably working on like ways for payment channels or credit assignment to propagate back upstream, different ways to think about the value and security of it.

Like the part about the paper and the presentation that was most interesting to me was these different economically grounded estimates, which differed by orders of magnitude, ultimately.

So it was a very restricted and humble analysis because, and then it didn't include Linux.

It included some packages on some services from some companies.

And so it's kind of like turning on the flashlight to use economically normal, normative estimation methods, different ways of triangulating what just the economic narrow value, like software with a price tag would be, let alone this training improvement of infrastructural type software.

So just, just that even when so restricted, still, there was a wide range of estimation.

So I guess how, how does estimation move forward?

And then what do those estimates, who do you tell about those estimates?

Or who do you, who do you, who do you want to see do what if they were to know that a given method of value assessment were this or that?

Mm hmm.

No, this is a great point.

I think part of, I think there are some efforts in, yeah, and government and when it comes, you know, to estimating GDP in general, and we would love to see.

And a version brought into the GDP, right?

Where you can estimate as a better value actually impacts GDP measures.

That would be fantastic.

Of course, this doesn't mean that what we have used here needs to be incorporated.

One has to really think about what's, you know, like an optimal version for that.

But also just generally speaking, I think provide, I think providing those different estimates is helpful for first providing one estimate.

I think it's helpful when you talk to anybody that has not even an idea, has not even heard about open source software, right?

There, some people may just go on the internet, they use some codes and they're happy that they found it and they are, you know, very new to the space.

And you can, you can pull people in with, hey, you know, this is a giant space and it's an extremely valuable space.

So that's one, even when you look at the lower bound estimate, I think I like to think about that as I had some work on vaccination prior to the pandemic.

And I had to explain, you know, to people the importance of that.

And I feel this is kind of very similar for open source software.

When you talk to people that are not in the space, they are not going to appreciate it as much.

And even some of the people that are, they just, you know, like you said there's a lot of free writing going on, which is okay.

But there's also some aspect of, you know, like there's a lot of value created and how can this be sustained?

And yeah, so I think informing everybody first about the value is important.

It's also important, not just for society as a whole, but also when you think about companies in particular managers,

there are some organizations that have actual like open source software units, maybe developers are using open source software tools.

But then when they're engaging in their work, they have to also justify to their managers, why are they doing what they're doing?

Right. And what kind of values they are creating?

And if they can't do that, it's kind of challenging to talk to their manager.

The manager will be like, I don't know what you did.

But I know exactly when we buy a product from Microsoft that cost me X dollars.

And yeah, so, and I don't know that, you know, like, what's the cost when we do this here with an open source software tool, right?

And finding the right comparison.

And I think that's where we should be going.

So that, yeah, open source, so that people can justify the usage of it within companies even more.

I think that would be great if this paper can just make a little bit of a dent here in the general perception and perceptions at the workplace around open source software as well.

Yeah. Yeah. I think that quote that you had there, the price is what you pay value is what you get.

And it's the value of open source software.

And then there's like this deeper psychology to paying or valuing what has been provided and the different sensitivity to losing what already is there as opposed to gaining what could be.

And all these different types of costs like time costs, tech burden, that those things come up in the industry setting and in the research setting.

So again, it's just sort of like opening up some paths to explore.

Well, if every firm had to redevelop just some JavaScript packages, we'd be talking about like the economic activity of the whole world.

So we know we can't do that. So, okay, so we're going to talk, then we're going to have some sharing of code.

So then what does that look like in some sort of extreme world where every firm develops every package and every operating system?

And like even the largest corporations can't develop a secure Linux kernel to there's one version of everything.

Somehow everyone knows how to find it. There's one package for every single thing.

So, where are we, where do we want to be? Where is the redundancy contributing to resiliency?

And to maintaining portfolios of different approaches, even if there's like multiple visualization packages for a given language.

It's sort of like, it's not necessarily wasteful.

But just to be able to assess that and then to say, well, here's, here's, if we were to move this way, this is, these are the kinds of costs that would and wouldn't show up on the accounting sheet of a firm.

These are the costs that society would or wouldn't incur over these different timescales.

Like here's how much this software would cost to buy today.

But then over the longer timescale, maybe there's less opportunities for participation in skill development or research.

And that has a vast cost as well.

So how does that get addressed in that sort of techno tragedy of the commons?

Yeah, I mean, also, I mean, obviously, we're not addressing this in the paper.

Those are very deep questions.

And I think it's, yeah, it's very challenging to think about.

Like, you know, let's say you have a bunch of packages that are all doing something that is very similar.

As you point out, is this necessarily wasteful?

Maybe not, because we can think about like this just as an evolutionary process, right?

Like we need a couple of projects or ideas out there.

And then some of them over time will survive and they will be selected by whatever you want to call the best.

In this case, maybe the one that just is in highest demand.

But they're more like there's some kind of lock in effect because people have used it for some things initially.

And then, you know, that's why this particular package will be used all the time.

So, yeah, so I think that's an evolutionary aspect to that question.

But I also agree on like sustainability of the environment that we need to find a way of, especially once, you know, their packages have frozen to prominence, substantially, whatever that means, right?

So, if there may be some kind of threshold of demand, then we have to think about can the community support this?

Or can there be company efforts?

Can there be new models on supporting the open source software environment?

Because those packages that are used substantially, they are contributing substantially to the value of open source software, not just on the individual side, but also from a company perspective, even though they are maybe not as engaged.

And so that's one of the really the bigger picture questions that I see is also like how to pull in companies that are currently benefiting so much.

And yeah, like so that they are paying part of the cost, but also getting something in return, even maybe even more than what they're getting right now by contributing to this ecosystem in a way, in a way where it stimulates simulation innovation that is potentially beneficial for them, right?

And you can't ask a firm to maximize profits and, you know, do engage in things for free, but there can be a profit motive in the long term and engagement and innovation.

Yeah.

Yeah.

I think that's right.

Right.

So this technical debt issue with, with the lock-in problem that probably exists is higher.

This kind of technical debt, when you think about closed form, like closed software, proprietary software that you're locked in and you maybe even have a harder time because you need maintainers that are, you know, you have only a limited set of individuals that are familiar with this particular software.

Well, in for open source software, you potentially have always new people that could come in and look at the software and yeah, you can adopt it more easily.

So you can switch more easily.

Fixes can be generated more quickly.

Just the whole idea of crowdsourcing.

So I think that, yeah, generally technical debt, as you would be, as you are saying is, is going, going to be lower with open source software.

You need to have the, I guess, like the right, like still services around open source software for developers to then actually engage in all of that type of work.

Right.

And I think there are companies out there.

There are people out there that are doing exactly that.

There are people out there that are, you know, you just have to, you know, work with the right people.

But like, I guess in terms of how that impacts the value of open source software.

It's a good question.

I would think it leads to a potential underestimate here again, right?

Because it's just so much more like easy, like you just have less of a burden here.

And so the value is automatically higher because you can switch more quickly from one innovation to another.

But what exactly that value is that is hard for me to say or like how to measure that.

But yeah, intuitively that would be again, contributing to the idea of underestimation.

Yeah.

Great point.

Yeah.

It reminds me of the economic estimates, like every window could be broken and then we would be adding value by just rebuilding.

So that's the kind of rebuild approach, which, and then that, that risk or like strategic value of like using an open schema or off sourcing or having the opportunity to do that.

Or having the opportunity for people to off source.

Uh, and then how to quantify like the value of optionality or of resilience to certain kinds of technical changes.

Or like if you're relying on a proprietary software, even for like a microscope in a lab or a simulation toolkit.

And then the company slows down or goes out of business or, or, you know, any number of other things.

Whereas like, if you have the source code, it'll never get worse than it is.

Maybe one of its dependencies could have like a future security issue, but you'll always have that local function.

And that's sort of what we need for the scientific reproducibility either way, or like in the neuroimaging, uh, community, it, there's a well-developed stack to talk about which versions of which software are used that, that Bradley and colleagues have shared.

Um, previously.

And also Pedro writes in the live chat, there is implicit trust given to companies that build in the open.

So that's value for the companies or for the labs who are able to put out this useful thing.

Um, uh, Bradley, if you have any other comments, otherwise, Manuel, I will look forward to hearing like, yeah. Okay.

What are your, what are your next moves?

What are exciting directions and, and how you're continuing this research or other projects?

Um, yeah, sure.

Oh, yeah.

Thank you.

Um, yeah, this is great.

Um, I, I really liked this discussion.

Um, in terms of the value of open source, South Africa, I think, you know, I think, you know, the value of open source software, what I would like to see, which is very difficult.

I would love to just see a map, uh, or have a map of usage, um, and maybe not just usage, but really then the value created, um, across the world.

I think that would be a, you know, ambitious endeavor.

That's very tough, uh, especially because you have to, you know, some of this information is proprietary.

And exactly in this instance here, we were not able to get such granular information, uh, where we could, where we were able to map everything to every region in the world.

But I think that would be a great descriptive starting point for an, an addition for this value of open source software project.

And just generally, um, figuring out more information on usage across the world, uh, within companies, um, what are they, which kind of packages are used when and where I think it's extremely valuable and important.

And yeah, so I think that's on the value of open source side.

Now I started a little bit talking about AI, um, get around that, but so now the value of open source software, one thing that we are pondering, uh, about it, um, what about the value of open source, um, or open source AI, if you like.

And we can, you know, think about what that concept actually is.

Yes, but, um, there's, of course, it's a similar discussion that was, that is around software generally right open, well, the proprietary, which, uh, I think is not has been basically.

Um, or is basically concluded almost where open source software one, and, um, now a similar discussion

exists in the space of AI, right?

Should it be open versus closed.

And, uh, so we would like to understand like, what's the value of open source AI relative to closed form alternatives.

Um, that would be, you know, in the spirit of this project, we would love to, um, better understand that as well. But I think there's also a lot of work even before thinking about AI, um, you know, on, on how to, um, yeah, better understand the value, maybe across industries even more.

Um, we did this just for the externally facing dataset.

If you can really dig into an internally facing dataset and are able to, um, show some of the estimates across industry, I think that would be fantastic.

But again, also that hinges on finding the rights, uh, organizations, companies that are willing to share some of that information that we need, uh, to then, um, engage in our value estimation.

And another one is, um, you know, sharing just, just the methodology broadly speaking, and hopefully the methodology being a different way.

The methodology being adopted more widely, even, um, you know, let's say on GitHub or any kind of open source, uh, platform where people could get a rough sense of an, uh, like value estimate.

And, and this won't be perfect by any means, but, uh, it's, it's something that, uh, at least it's a little bit more tangible.

Uh, so that would be great, uh, great too.

Um, yeah.

So I think that's, uh, what we're doing in terms of, um, better understanding the value of open source software.

We have also, um, other projects in the space of open source software.

Um, in particular, we have a project that is more in general AI and better understanding, um, GitHub co-pilot and how it affects the nature of work for individuals.

That's, that's, that's, I think an exciting project, which is not as, you know, closely tied to this project here, but where we just try to see how work patterns are changing.

When you have access to, um, just generative AI, um, coding tool.

And we see that people seem to engage in more coding, this, uh, kind of project management activity.

And we see a lot of like kind of experimentation, um, out there, a little bit less of exploration and can put, uh, at the end of the day, um, yeah, we see that people are gauging more autonomous and less collaborative work.

Um, because they, you know, like they can use the AI tool instead of, um, engaging with the community where, um, which we like to call a project management is really engaging with, uh, other developers on the issues about, and then assigning other individuals.

And, um, yeah, at the end of the day, um, in terms of experimentation, you, um, we find that people are engaging more, um, exposing themselves more to new languages that are more, whereas more in the marketplace.

And we were able to use that to, to put a dollar value kind of also very much like, uh, back of the envelope

calculation to, um, get up co-pilot, uh, coding AI.

And we just have a couple of billion dollars, uh, just for exposing yourself to new languages.

So yeah, we always try to think about, you know, the value that is generated, um, for the community or when the community interacts, um, with tools, with open source, uh, or with, with tools that are intersecting with the open source, uh, software space.

So, yeah, I think that's super exciting and, and happy to chat with anybody or, um, also emails, uh, are more than welcome and any questions.

Thank you.

Thank you.

Thank you.

I think maybe in the future we will hear more from your research and a scientific open source meta analysis, Bradley and or other in, uh, the audience that could be very interesting to trace back some of those and, and just talk about it from a scientific computing value chain rather than the industrial.

Yeah.

Sure.

Sounds good.

Yeah.

That's great to you.

Cool.

Thank you.

Appreciate it.

See you next time.

You too.

Thank you so much.

Thank you.

Bye.

Bye.

Bye.